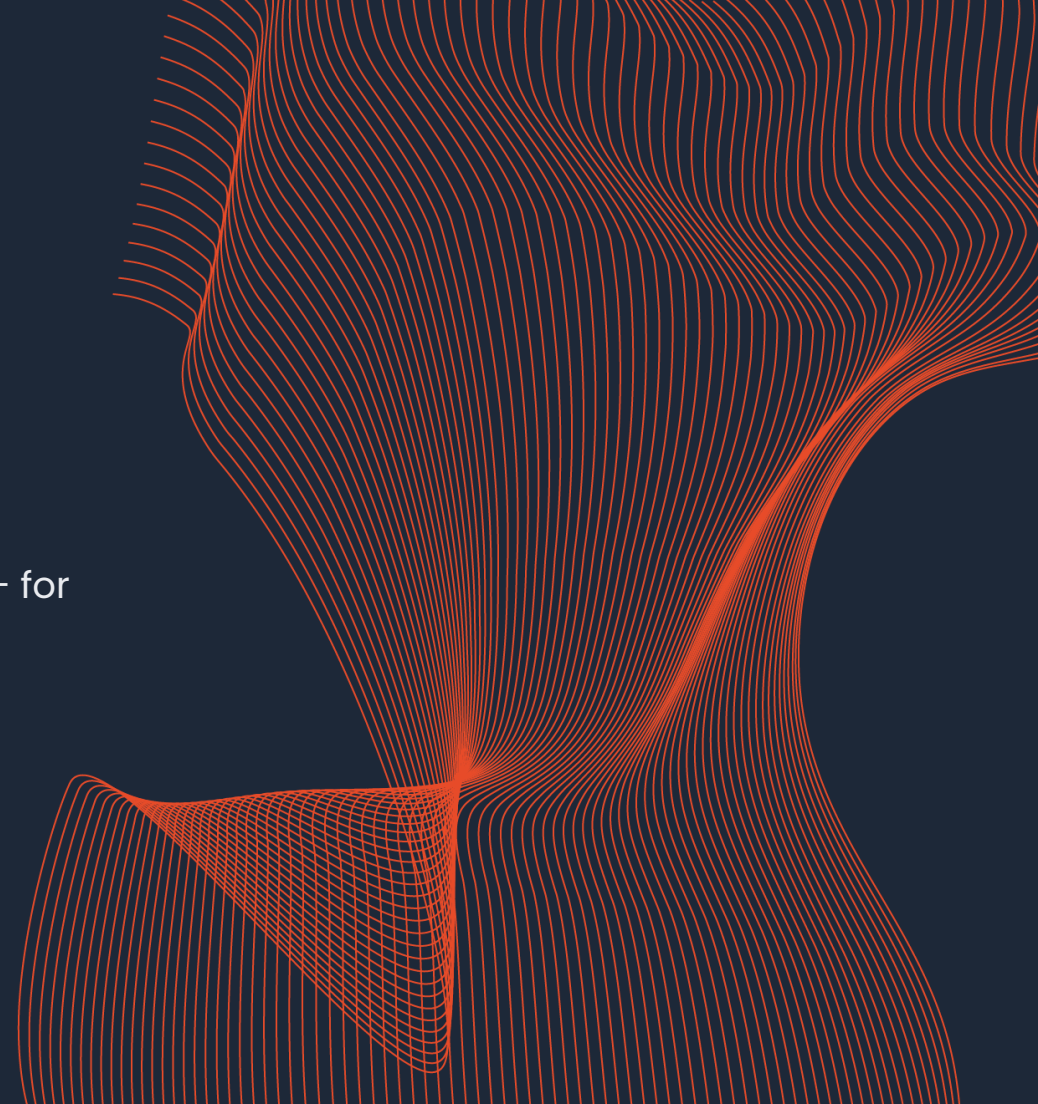
theodo.

CSS-First Best Practices

Doing more with **CSS** and less **JavaScript** — for performance, SEO & maintainability.

El Mehdi Assamer · Theodo



theodo.

THE IDEA

Why CSS-first?

The browser already knows how to animate, lay out and handle interactions natively. **CSS-first means using those built-in powers before writing JavaScript** for lighter, faster and more accessible pages.

What we gain by reaching for CSS first

⚡ Performance

- CSS animations use the **GPU**, so they stay smooth, even while JavaScript is busy
- Less JavaScript means **less to download and run**
- The page feels faster and never freezes

🔍 SEO

- Less JavaScript → pages **load faster**
- Faster sites **rank higher** on Google
- Content written in HTML/CSS is **easy for Google to read**

♿ Accessibility

- Native elements ship keyboard + ARIA **for free**
- `:focus-visible` handles keyboard users correctly

🧹 Maintainability

- Fewer dependencies to update & secure
- Declarative, the browser does the work
- Less code to test and debug

> **The rule of thumb:** if the browser can do it natively, don't ship JavaScript for it.

theodo.

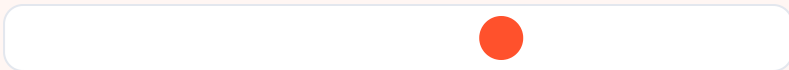
6 TECHNIQUES · LIVE BAD → GOOD

What we'll cover

1 · **@keyframes & transition** vs JS libraries
2 · **scroll-snap** carousel
3 · **:hover / :focus** micro-interactions
4 · **<details>** native accordion
5 · **Flexbox** layout
6 · **position: sticky**

1 · Animations: `@keyframes` over JS libraries

✕ JS ANIMATION LIBRARY



```
const ballLeftPercent = ref(0)
let frameId, direction = 1, position = 0

const moveBall = () => {
  position += direction * 1.4
  if (position > 100 || position < 0) direction *= -1
  ballLeftPercent.value = position          // → el.style.left
  frameId = requestAnimationFrame(moveBall)
}
onMounted(() => { frameId = requestAnimationFrame(moveBall) })
onUnmounted(() => cancelAnimationFrame(frameId))
```

Runs on the main thread · animates `left` → layout + paint every frame · ships a JS bundle

✓ PURE CSS



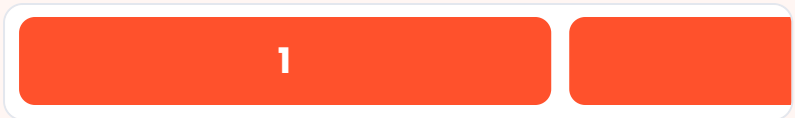
```
.ball--css {
  animation: slideXsafe 1s ease-in-out infinite alternate;
}
@keyframes slideXsafe {
  from { left: 0;    transform: translateX(0); }
  to   { left: 100%; transform: translateX(-100%); }
}
```

Runs on the compositor thread · animates `transform` → no layout · 0 KB JS

Best practice: animate `transform` & `opacity` in CSS they run on the GPU/compositor. Reach for a JS library only for physics, gestures or timeline-orchestrated sequences.

2 · `scroll-snap`: a carousel without JavaScript

✕ JS CAROUSEL LIBRARY



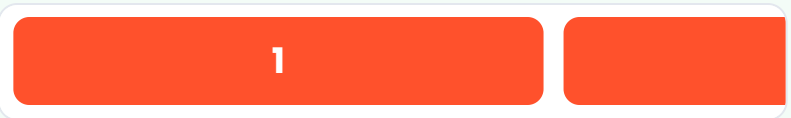

```
.rail--bad { display: flex; overflow-x: auto; }

/* stops mid-slide → you reach for a library */
import Swiper from 'swiper'
new Swiper('.rail', {
  slidesPerView: 'auto',
  centeredSlides: true,
  // + CSS bundle, breakpoints, event wiring…
})
```

Free-scrolls and stops **mid-slide** unless JavaScript forces the snap.

Extra bundle.

✓ CSS SCROLL-SNAP




```
.rail--good {
  display: flex;
  overflow-x: auto;
  scroll-snap-type: x mandatory;
  scroll-behavior: smooth;
}
.rail--good .slide { scroll-snap-align: center; }
```

Drag/scroll → the browser **snaps to each slide**. Zero JavaScript.

Best practice: for sliders & carousels, use `scroll-snap-type` on the scroller and `scroll-snap-align` on each item. The browser snaps to each slide no carousel library needed.

3 · `:hover` / `:focus` micro-interactions

✕ JS EVENT HANDLERS

Hover me

```
const hovered = ref(false)

<button
  @mouseenter="hovered = true"
  @mouseleave="hovered = false"
  :style="{ transform: hovered ? 'translateY(-4px)' : 'none',
            boxShadow: hovered ? '0 10px 20px …' : 'none' }">
```

Extra JS, no keyboard state, and `:focus` is forgotten.

✓ PURE CSS STATE

Hover / Tab to me

```
.btn--css { transition: transform .2s ease, box-shadow .2s ease; }
.btn--css:hover {
  transform: translateY(-4px);
  box-shadow: 0 10px 20px rgba(255, 81, 44, .35);
}
.btn--css:focus-visible { outline: 3px solid var(--th-navy); }
```

Zero JS · keyboard users get a `:focus-visible` ring for free.

Best practice: express interactive states with CSS pseudo-classes, not JS events. Always pair `:hover` with `:focus-visible` so keyboard users get the same affordance.

4 · `<details>`: a native, accessible accordion

✕ DIV + JS CLICK HANDLER

What is CSS-first? +

```
const open = ref(false)

<div class="acc__head" @click="open = !open">
  What is CSS-first? <span>{{ open ? '−' : '+' }}</span>
</div>
<div class="acc__body" v-show="open">…</div>
<!-- still missing: aria-expanded, role, keyboard… -->
```

✓ NATIVE <DETAILS>

What is CSS-first? +

```
<details>
  <summary>What is CSS-first?</summary>
  <div>Open/close, keyboard & ARIA built in.</div>
</details>
```

Best practice: use `<details>/<summary>` for disclosure UI. Open/close, keyboard and ARIA are built in, and the content stays in the DOM — **visible to search engines**.

5 · Flexbox over manual positioning

✕ POSITION: ABSOLUTE + MANUAL OFFSETS

Short | A bit more text … | Tiny

```
.row { position: relative; height: 40px; /* hardcoded */ }
.col { position: absolute; top: 4px; width: 32%; }
.col:nth-of-type(1) { left: 4px; }
.col:nth-of-type(2) { left: 0; margin-left: 34%; }
.col:nth-of-type(3) { right: 4px; }
```

Out of flow → parent **collapses**, so you hardcode a height; positions are **manual math** and content overflows.

✓ DISPLAY: FLEX

Short | A bit more text in this column | Tiny

```
.row { display: flex; gap: .6rem; }
.col { flex: 1; }
```

Equal heights automatically, even spacing with `gap`, no clearfix.

Best practice: use **Flexbox** for rows & columns. `display:flex` gives equal-height columns, `gap` spaces them, and `justify/align` centre content no absolute coordinates, no hardcoded heights.

6 · `position: sticky` over scroll listeners

✕ JS SCROLL LISTENER → POSITION: FIXED

Header (toggled by JS)

Scroll line 1 — the handler fires on every scroll event.

Scroll line 2 — the handler fires on every scroll event.

```
window.addEventListener('scroll', () => {
  if (window.scrollY > header.offsetTop) {
    header.classList.add('is-fixed')    /* position: fixed */
  } else {
    header.classList.remove('is-fixed')
  }
})
```

Fires constantly, causes layout thrash & jank, needs debouncing.

✓ POSITION: STICKY

Header (sticky)

Scroll line 1 — the browser pins it natively.

Scroll line 2 — the browser pins it natively.

```
.header {
  position: sticky;
  top: 0;
}
```

No listeners, no JS, buttery-smooth — handled by the compositor.

Best practice: pin headers, sidebars and section labels with `position: sticky; top: 0`. No scroll listeners, no layout thrash the browser handles it on the compositor.

Bonus · Good CSS habits

🎨 **Use variables for your design tokens** — change a colour once, everywhere updates.

```
:root { --brand: #ff512c; }
.btn  { background: var(--brand); }
```

📏 **Prefer relative units** — scales & stays accessible.

```
font-size: clamp(1rem, 2.5vw, 1.5rem); /* not 24px */
```

🧱 **Space with `gap`, not margins** — no fragile margin hacks.

```
.list { display: flex; gap: 12px; }
```

🚫 **Avoid `!important`** — lower specificity instead.

```
.title { color: red !important; } /* ✗ */
.card .title { color: red; }      /* ✓ */
```

🪜 **Keep selectors shallow** — deep nesting is brittle.

```
.nav ul li a span { }  /* ✗ */
.nav-link { }          /* ✓ */
```

♿ **Respect user settings** — wrap motion in a media query.

```
@media (prefers-reduced-motion: reduce) {
  * { animation: none; transition: none; }
}
```

> **Golden rule:** write the simplest CSS that works, the browser is smarter than you think.

theodo.

TAKEAWAYS

Reach for the platform first

Animate with `@keyframes` / `transition` on `transform` & `opacity`

Carousels → `scroll-snap`, not a JS library

State belongs in `:hover` / `:focus-visible`, not JS events

Disclosure UI → native `<details>`

Layout → Flexbox, not floats

Pinning → `position: sticky`

> Less JavaScript → faster pages → better SEO → happier users. **That's the win.**

theodo.

Thank you 🧡

